CPSC 304 Project Cover Page

Milestone #: 4

Date: August 11, 2026

Group Number: 41

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
George Yuan	11195369	s0w9l	gyuanedu@gmail.com
Adrian Wong	79387718	o3v5f	adrianwong200@gmail.com
Sam Dittmar	47583216	t6f7i	sam.dittmar@outlook.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above. (In the case of Project Milestone 0, the main purpose of this page is for you to let us know your e-mail address, and then let us assign you to a TA for your project supervisor.)

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

Description:
Our project is an online admin tool for managing a community garden. Users can manage detailed planting records for individual beds in the plots of the garden and can search for beds by various details to find the bed best suited for their planting needs. The app allows users to inspect specific details of the water schedules of the garden plots and manage information about the tool sheds around the garden. The app reports profit per harvest day, and provides profit analysis for specific crops per harvest. The app provides a suite of other administrative properties such as equipment condition level monitoring and gardener plot assignment management.

Changes:
Removed participation constraint on PlotID's relationship because it seems more feasible for Plots to contain at most one Shed, and to be able to update ShedID's PlotID in the Update query.

Changed all attributes with a BOOLEAN domain type to Integer where 0 is false and 1 is true, because BOOLEAN was not a valid data type.

Assertions and Triggers:
Participation constraint on Harvest in Taken
Participation constraint on Harvest in Logs
Participation constraint on PlantingRecord in Records
Participation constraint on WaterSchedule in AssignedTo
Participation constraint on WaterSchedule in Oversees

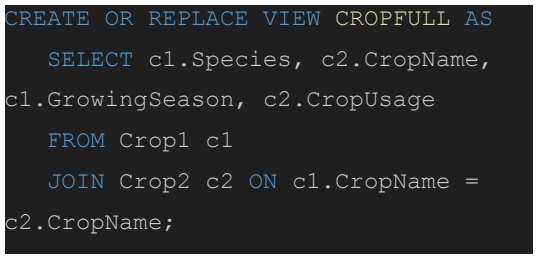
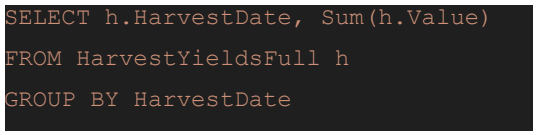
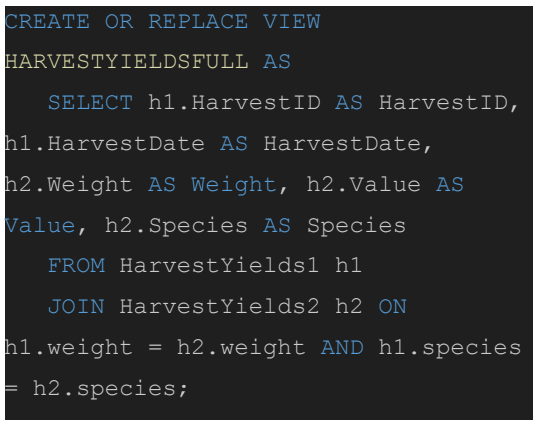
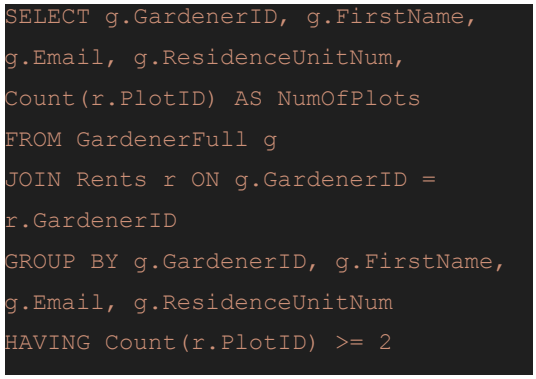
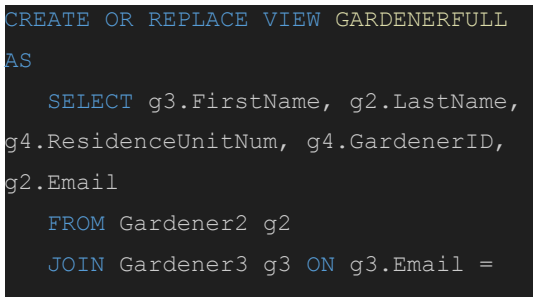
Table of Queries

Query Type	SQL Statement	Location in Code	Function Name
INSERT	`SELECT BedID, PlotID, FertilizerType, Width, Length, Height FROM BedContains WHERE (BedID, PlotID) NOT IN (SELECT BedID, PlotID FROM PlantingRecordTracks`	appService.js at line 78	fetchRectableFromDb (Only so users can see a successful insert)
	`INSERT INTO PlantingRecordTracks ( PlantingID, PlantingStyle, QuantityPlanted, PlantedDate, ExpectedHarvestDate, BedID, PlotID )`	appService.js at lines 97-107	insertRectable (Use for grading)

	VALUES (:id, :style, :amount, TO_DATE(:plantDate, 'YYYY-MM-DD'), TO_DATE(:harvDate, 'YYYY-MM-DD'), :bedID, :plotID) SELECT BedID, PlotID, FertilizerType, Width, Length, Height FROM BedContains WHERE (BedID, PlotID) NOT IN (SELECT BedID, PlotID FROM PlantingRecordTracks	appService.js at lines 133-136	fetchAvBedTableFromDb (Only so users can see available FKs)
DELETE	SELECT Email FROM GARDENER4 WHERE GardenerID = :gardenerID` SELECT PlotID FROM RENTS WHERE GardenerID = :gardenerID DELETE FROM GARDENER4 WHERE GardenerID = :gardenerID DELETE FROM GARDENER3 WHERE Email = :email DELETE FROM GARDENER2 WHERE Email = :email UPDATE PLOT SET IsAvailable = 1 WHERE PlotID = :plotID	appService.js at lines 636, 648, 654, 660, 666, 674	deleteGardener (Only to check if such gardener exists) (Only to update Plot's IsAvailable attribute) (Use for grading) (Use for grading) (Use for grading) (Only to update Plot's IsAvailable attribute)
UPDATE	SELECT * FROM ToolShedLocatedAtFull SELECT * FROM Plot WHERE PlotID NOT IN (SELECT PlotID FROM ToolShedLocatedAt2) SELECT COUNT(*)	appService.js at lines 225-226 appService.js at lines 237-240 appService.js	fetchToolShedLocatedAtTableFromDb (Only so users can see a successful update) fetchAvPlotTableFromDb (Only so users can see available FKs) updateCapacityToolSh

	FROM ToolShedLocatedAt2 WHERE ShedID=:toolShedID UPDATE ToolShedLocatedAt2 SET Capacity=:newCapacity WHERE ShedID =:toolShedID	at lines 252-254, 265-267	edLocatedAtTable (Use for grading) (Use for grading)
	SELECT COUNT(*) FROM ToolShedLocatedAt2 WHERE ShedID=:toolShedID UPDATE ToolShedLocatedAt2 SET NumTools=:newNumTools WHERE ShedID =:toolShedID	appService.js at lines 287-289, 300-302	updateNumToolsToolShedLocatedAtTable (Use for grading) (Use for grading)
	SELECT COUNT(*) FROM ToolShedLocatedAt2 WHERE ShedID=:toolShedID UPDATE ToolShedLocatedAt1 SET ConditionLevel=:NewConditionLevel WHERE LastCleanDate = (SELECT LastCleanDate FROM ToolShedLocatedAt2 WHERE ShedID =:toolShedID)	appService.js at lines 325-327, 338-342	updateConditionLevelToolShedLocatedAtTable (Use for grading) (Use for grading)
	SELECT COUNT(*) FROM ToolShedLocatedAt2 WHERE ShedID=:toolShedID UPDATE ToolShedLocatedAt2 SET PlotID=:NewPlotID WHERE PlotID=:OldPlotID	appService.js at lines 362-364, 375-377	updatePlotIDToolShedLocatedAtTable (Use for grading) (Use for grading)
	CREATE OR REPLACE VIEW TOOLSHEDLOCATEDATFULL AS	tablesMain.sql at lines 311-314	(Only to make the view ToolShedLocatedAtFul

	SELECT t2.ShedID AS ShedID, t1.LastCleanDate AS LastCleanDate, t2.Capacity AS Capacity, t2.NumTools AS NumTools, t1.ConditionLevel AS ConditionLevel, t2.PlotID AS PlotID FROM ToolShedLocatedAt1 t1 JOIN ToolShedLocatedAt2 t2 ON t1.LastCleanDate = t2.LastCleanDate;		l)
SELECTION	SELECT BedID, PlotID, FertilizerType, Length, Width, Height FROM BedContains	appService.js at lines 133-136	selectBedsFromDb (Use for grading)
PROJECTION	SELECT ${sqlString} FROM WATERSCHEDULEFULL CREATE OR REPLACE VIEW WATERSCHEDULEFULL AS SELECT w1.SCHEDULEID, w1.FREQUENCY, w1.STARTTIME, w1.TEMP, w2.AMOUNT FROM WATERSCHEDULE1 w1 JOIN WATERSCHEDULE2 w2 ON w1.STARTTIME = w2.STARTTIME AND w1.FREQUENCY = w2.FREQUENCY;	appService.js at lines 207-208 tablesMain.sql at lines 306-309	projectScheduleFromDb (Use for grading) (Only to make the view ToolShedLocatedAtFull)
JOIN	SELECT COUNT(*) FROM Crop1 WHERE CropName=:cropName SELECT h.HarvestDate, h.Weight, h.Value, c.CropName FROM HARVESTYIELDSFULL h JOIN Crop1 c ON c.Species = h.Species WHERE c.CropName =:cropName SELECT * FROM CROPFULL	appService.js at lines 403-405, 416-419 appService.js at lines 445-446	joinHarvestYieldsAndCropGivenCropName (Only to see if the cropName exists) (Use for grading) (Only so users can see available CropNames)

		tablesMain.sql at lines 321-324	(Only to make the view CropFull)
Aggregation with Group By	 	appService.js at lines 390-392 tablesMain.sql at lines 316-319	sumHarvestTableFromDb (Use for grading) (Only to make the view HarvestYieldsFull)
This query shows the total value of all harvests when grouped by harvest date.			
Aggregation with Having	 	appService.js at lines 456-460 tablesMain.sql at lines 326-330	gardenersMoreThanOnePlotTableFromDb (Use for grading) (Only to make the view GardenerFull)

	g2.Email JOIN Gardener4 g4 ON g4.Email = g2.Email;		
This query shows each gardener who rents at least 2 plots, along with their gardenerID, first name, email, residence unit number, and the number of rented plots			
Nested Aggregation with Group By	SELECT s1.ShedID, s1.EquipmentType, AVG(s1.ConditionLevel) AS avgConditionLevel FROM StoresAndCheckedOutEquipmentFull s1 GROUP BY s1.ShedID, s1.EquipmentType HAVING AVG (s1.ConditionLevel) <= (Select AVG (s2.ConditionLevel) FROM StoresAndCheckedOutEquipmentFull s2 WHERE s1.EquipmentType = s2.EquipmentType GROUP BY s2.EquipmentType) ORDER BY ShedID ASC, EquipmentType ASC	appService.js at lines 471-478	lowestAvgEquipmentConditionByShedTableFromDb (Use for grading)
	CREATE OR REPLACE VIEW STORESANDCHECKEDOUTEQUIPMENTFULL AS SELECT s2.EquipmentID, s1.PurchaseDate, s2.EquipmentType, s1.ConditionLevel, s2.ShedID, s2.GardenerID FROM StoresAndCheckedOutEquipment1 s1 JOIN StoresAndCheckedOutEquipment2 s2 ON s2.PurchaseDate = s1.PurchaseDate;	tablesMain.sql at lines 332-335	(Only to make the view StoresAndCheckedOutEquipmentFull)
This query shows the equipment type in each shed whose average condition is less than or equal to the overall average condition level for that equipment type across all sheds.			
Division	SELECT g4.gardenerid, g3.firstname, g2.lastname	appService.js at lines	lfetchGardenerRentingAllPlotsOfType (Use

	FROM GARDENER4 g4 JOIN GARDENER2 g2 ON g4.email = g2.email JOIN GARDENER3 g3 ON g4.email = g3.email WHERE g4.gardenerid IN (SELECT r1.gardenerid FROM RENTS r1 WHERE NOT EXISTS ((SELECT plotid FROM PLOT WHERE isindoor = :isindoor) MINUS (SELECT r2.plotid FROM RENTS r2 WHERE r2.gardenerid = r1.gardenerid))) ORDER BY g4.gardenerid	696-709	for grading)
This query shows which gardeners are renting either all of the available indoor plots or outdoor plots.			

Screenshots:

Insert:



Manage Planting Records

▼ Click to open

ID	Style	Amount	Date Planted	Expected Harvest Date	BedID	PlotID
1	Hill	10	7/30/2026	9/30/2026	1	1
2	Row	12	8/1/2026	9/1/2026	2	2
3	Strip	1	3/17/2026	4/15/2026	3	3
4	Broadcast	23	5/24/2026	10/23/2026	4	4
5	Raised Bed	5	3/10/2026	4/17/2026	5	5

Create new planting record

ID:
6

How will you plant it?:
With care

Amount:
67

Date Planted:
2026-01-01

Exp Harvest Date:
2026-12-31

BedID:
2

PlotID:
1

insert

Available Beds

BedID	PlotID	Fertilizer	Width	Length	Height
2	1	Magic	10	10	5

▼ Click to open

ID	Style	Amount	Date Planted	Expected Harvest Date	BedID	PlotID
1	Hill	10	7/30/2026	9/30/2026	1	1
2	Row	12	8/1/2026	9/1/2026	2	2
3	Strip	1	3/17/2026	4/15/2026	3	3
4	Broadcast	23	5/24/2026	10/23/2026	4	4
5	Raised Bed	5	3/10/2026	4/17/2026	5	5
6	With care	67	12/31/2025	12/30/2026	2	1

Create new planting record

ID:
6

How will you plant it?:
With care

Amount:
67

Date Planted:
2026-01-01

Exp Harvest Date:
2026-12-31

BedID:
2

PlotID:
1

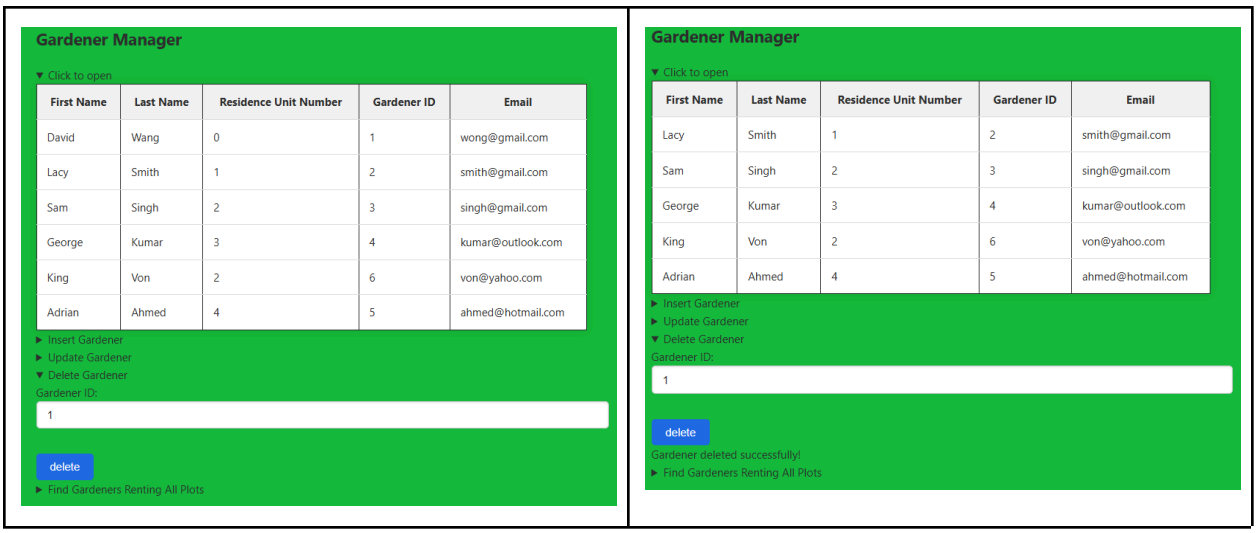
insert

Data inserted successfully!

Available Beds

BedID	PlotID	Fertilizer	Width	Length	Height

Delete:



Gardener Manager

▼ Click to open

First Name	Last Name	Residence Unit Number	Gardener ID	Email
David	Wang	0	1	wong@gmail.com
Lacy	Smith	1	2	smith@gmail.com
Sam	Singh	2	3	singh@gmail.com
George	Kumar	3	4	kumar@outlook.com
King	Von	2	6	von@yahoo.com
Adrian	Ahmed	4	5	ahmed@hotmail.com

▶ Insert Gardener
▶ Update Gardener
▼ Delete Gardener
Gardener ID:
1

delete

▶ Find Gardeners Renting All Plots

Gardener Manager

▼ Click to open

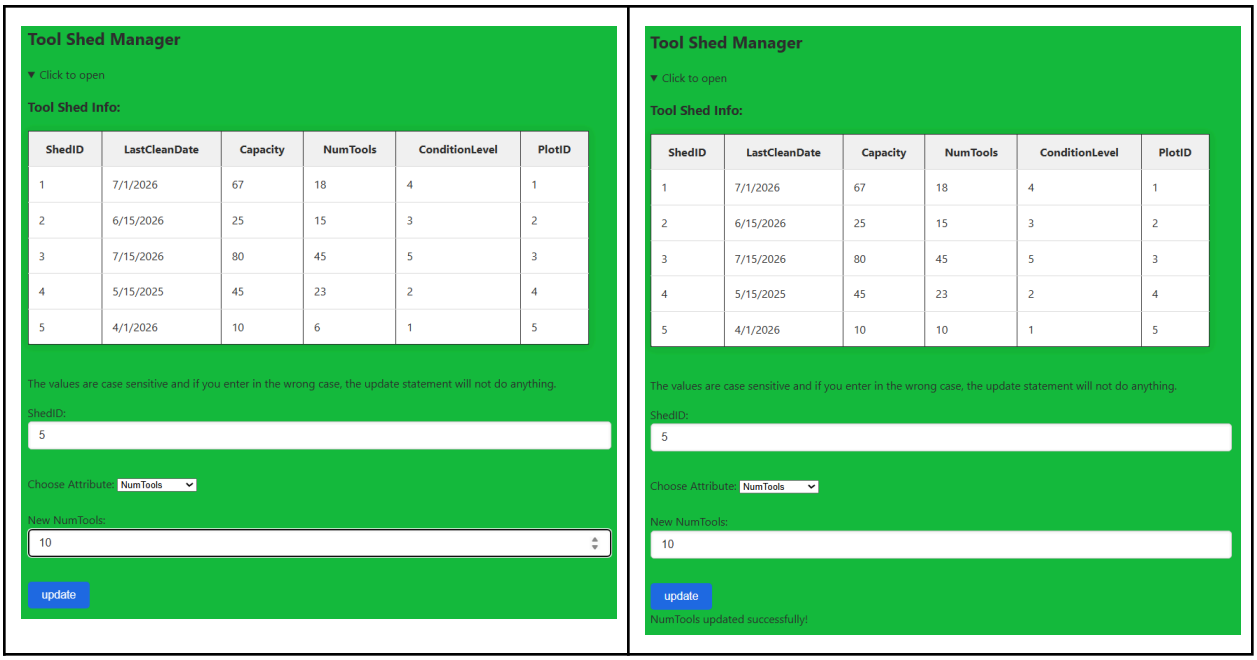
First Name	Last Name	Residence Unit Number	Gardener ID	Email
Lacy	Smith	1	2	smith@gmail.com
Sam	Singh	2	3	singh@gmail.com
George	Kumar	3	4	kumar@outlook.com
King	Von	2	6	von@yahoo.com
Adrian	Ahmed	4	5	ahmed@hotmail.com

▶ Insert Gardener
▶ Update Gardener
▼ Delete Gardener
Gardener ID:
1

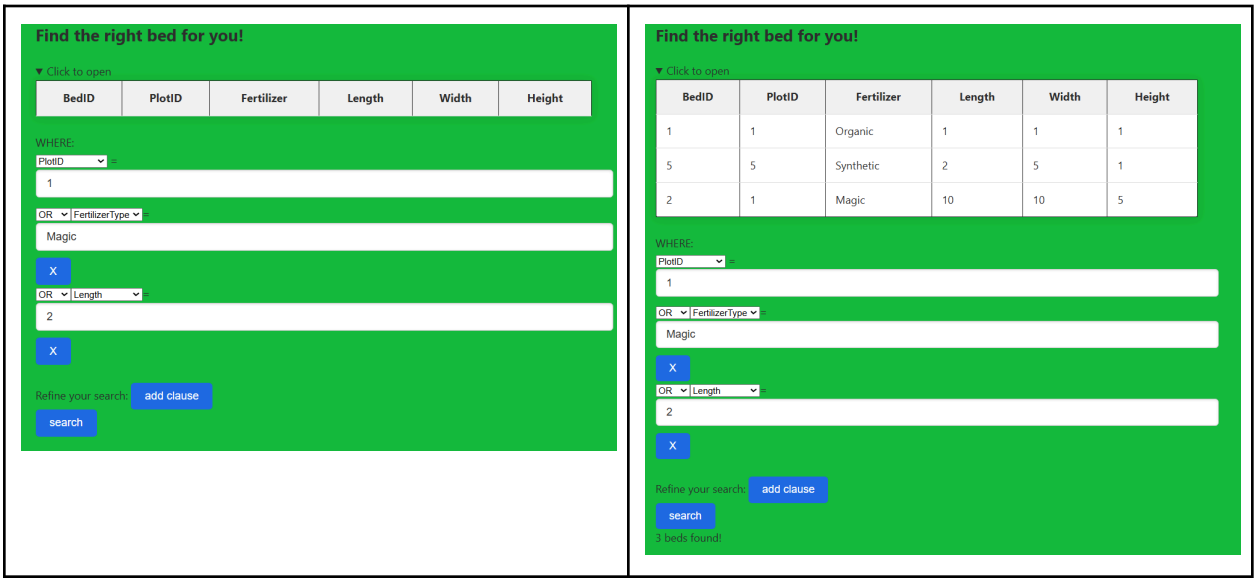
delete

Gardener deleted successfully!
▶ Find Gardeners Renting All Plots

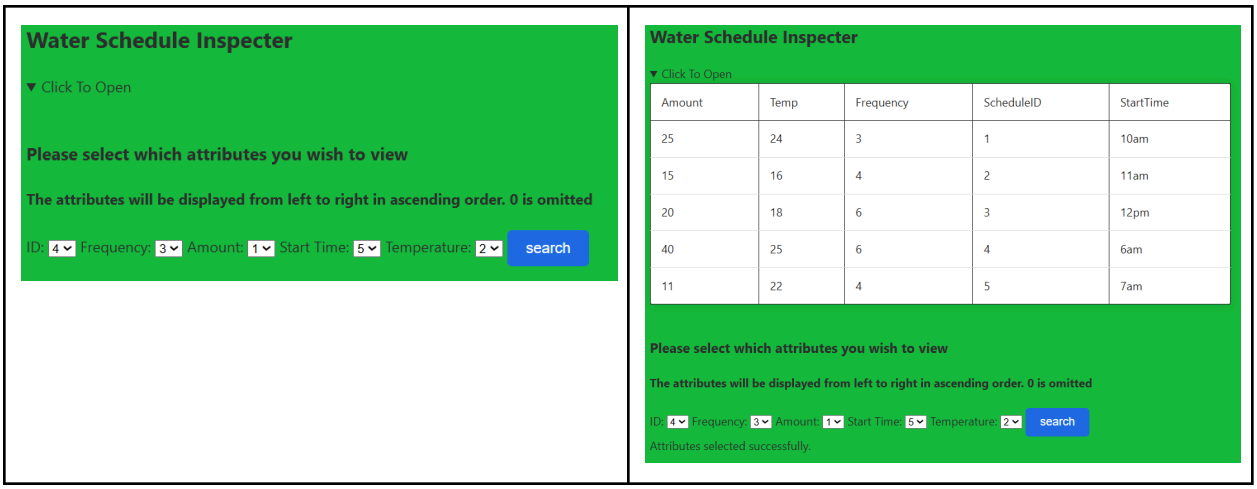
Update:



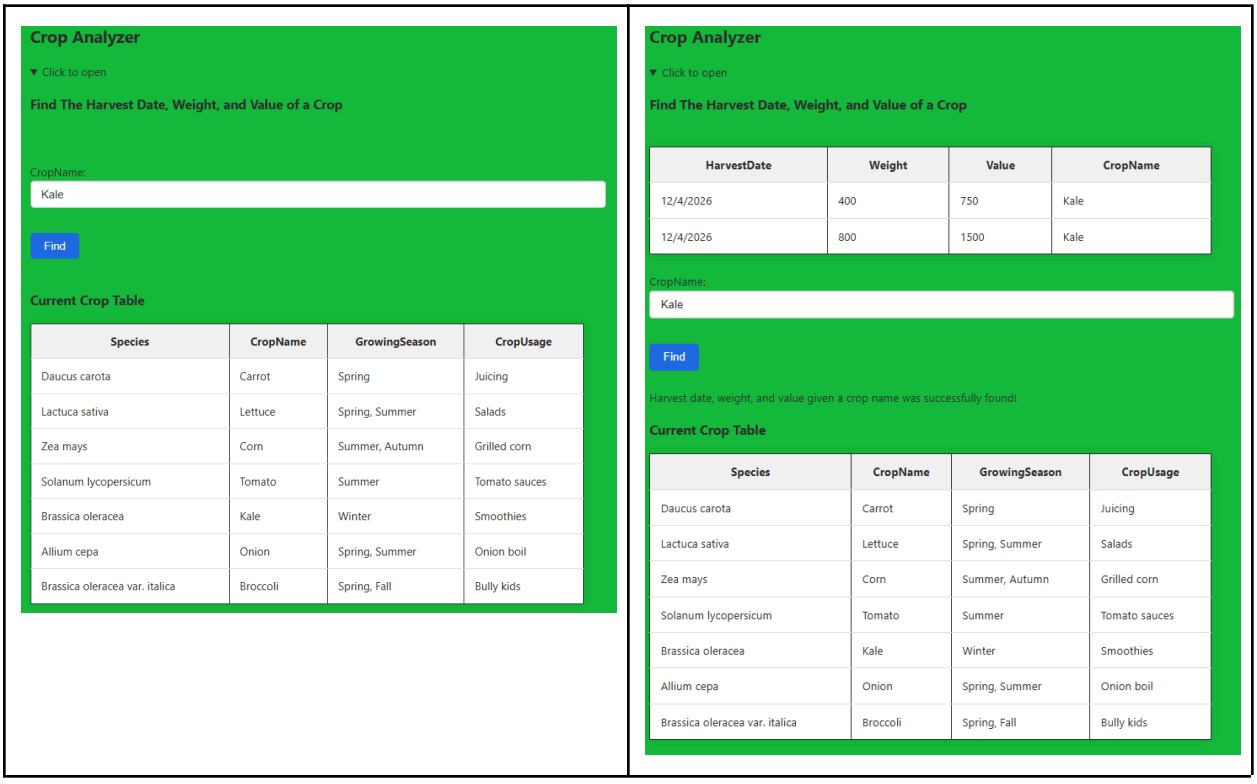
Selection:



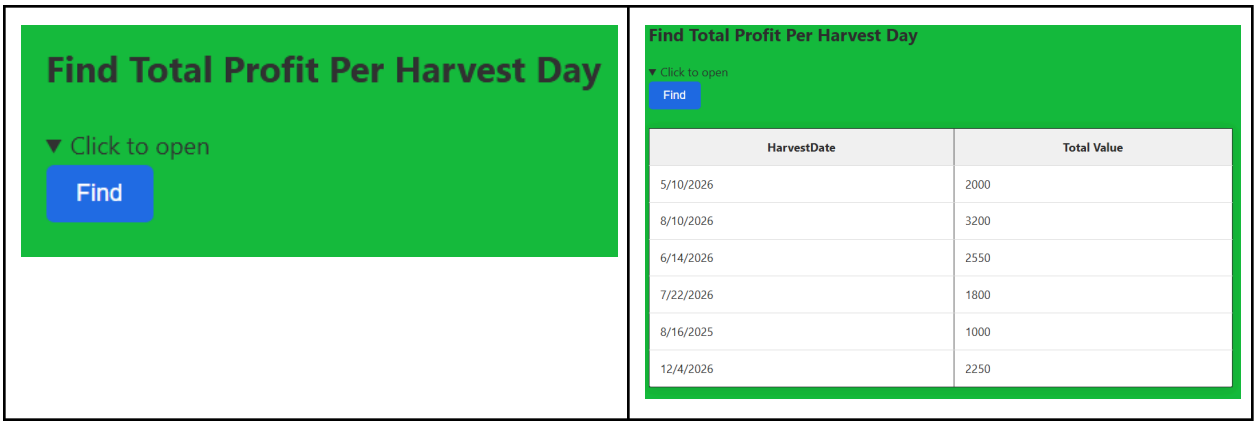
Projection:



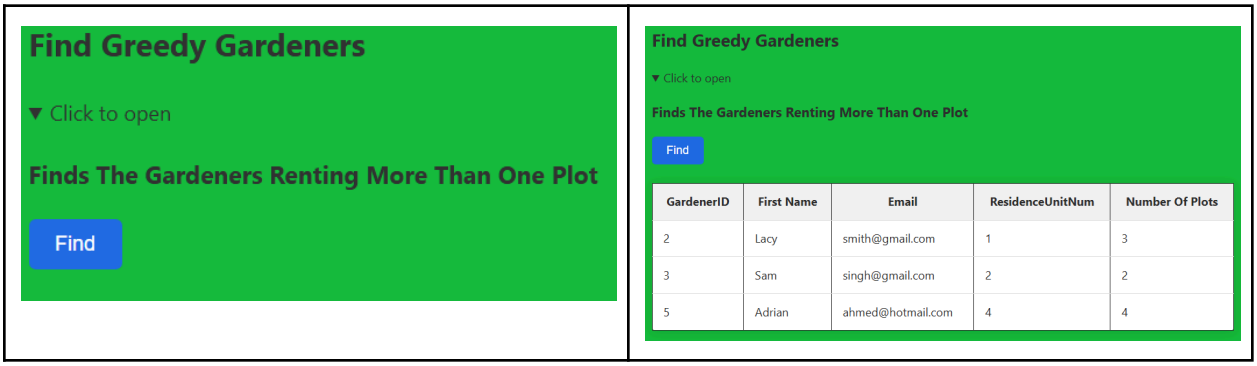
Join:



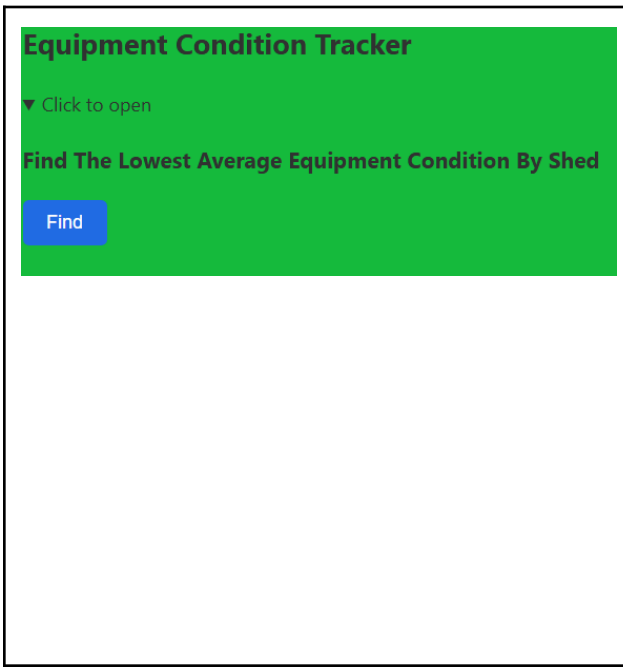
Aggregation with Group By:



Aggregation with Having:

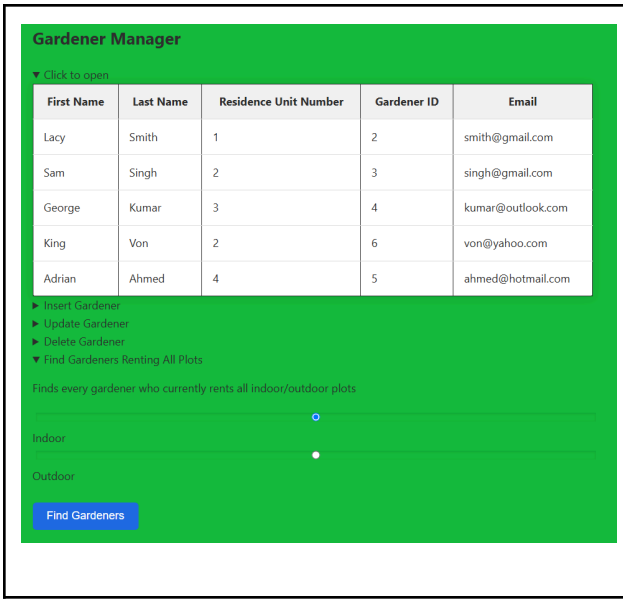


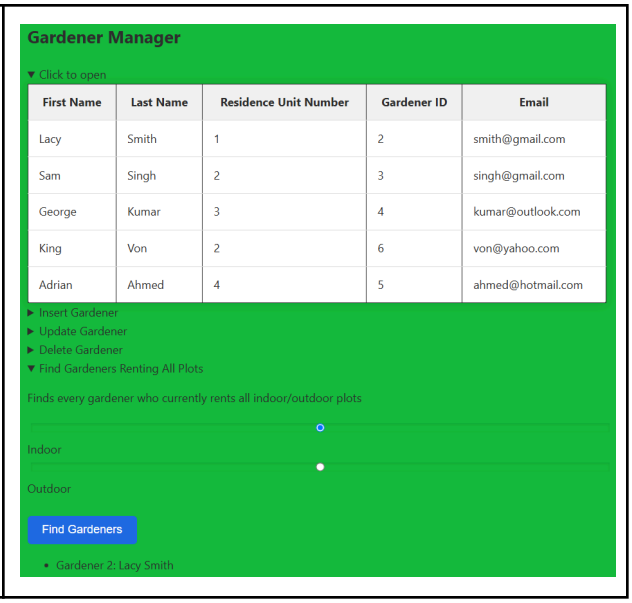
Nested Aggregation with Group By:





Division:





Submit the URL of your repository with all of the required items in it:

https://github.students.cs.ubc.ca/CPSC304-2026S/team_41.git

GitHub account

You must use your UBC GitHub account for all commits to your repository.

- Used personal email: gyuanedu@gmail.com to commit (CWL: gyuan03). Moving forward I understand that I will use my UBC account.